KnowWhen-R

Activation probes for answering, retrieving and abstaining

Lexsi AI Safety and Alignment Hackathon 2026
Track 2b, problem statement 13

November 2026

Abstract

Small open language models answer many factual questions wrongly, and they sound just as confident when they are wrong as when they are right. Problem statement 13 asks for probes on internal activations that flag likely errors, so that the model can abstain or retrieve, giving fewer wrong answers while most questions are still answered. We built this for two frozen Cohere Labs models, Tiny Aya Global and Aya Expanse 8B. Three linear probes read the hidden states: one judges the closed book answer, one forecasts whether a retrieved answer would be right, and one judges the retrieved answer. On 3,968 held out test questions, answering everything from memory left 79.2% of questions wrong on Tiny Aya and 65.8% on Aya Expanse. With exactly 70% of questions answered, our routed system left 29.0% and 23.1% wrong, against 49.7% and 37.1% for a closed book probe that abstains. The gain also holds on each of four datasets left out of training. The results have clear limits, and we report them: most of the gain comes from retrieval plus the probe on the retrieved answer, routing adds only 1 to 3 points over always retrieving, and on closed book answers from unseen datasets simple token statistics often match the activations. The English trained probe also transfers to five other languages with no retraining. A label free online cut keeps the share answered within about 2 points of the target on unseen datasets. This report also describes what we tried that did not work and the bugs we found and fixed along the way.

1. The question

The problem statement asks us to train probes on internal activations to flag likely hallucinations, so that the model abstains or triggers retrieval when a probe fires, and to show fewer wrong answers while the model still answers most questions.

We read that last sentence as the success test. A system that abstains on nearly everything is safe and useless, so every headline number below gives the wrong answer rate together with the share of questions answered. Wrong answer rate here always means wrong answers divided by all questions, including the ones the system declined.

We asked three questions. Do probes keep working on a dataset they never saw (H1)? Does a decision rule that prices errors and retrieval beat fixed thresholds (H2)? Can we get fewer wrong answers without retrieving for every question (H3)?

2. Main result

Table 1 is the before and after on the 3,968 test questions, with every system set to answer exactly the same share of questions. Figure 1 shows the full curves.

At 70% answered, routing removes about 40% of the wrong answers left by a closed book probe, on both models. The share of correct answers also rises, from 20.2% to 41.0% on Tiny Aya and from 32.9% to 46.9% on the 8B model, so this is not just abstention moving errors around.

Table 1: Wrong answers as a percentage of all test questions, at a fixed share of questions answered.

System	Tiny Aya 70%	Tiny Aya 90%	Aya 8B 70%	Aya 8B 90%
Closed book, answer everything (100% answered)	79.2	79.2	65.8	65.8
Always retrieve, answer everything (98% answered)	56.6	56.6	51.9	51.9
Closed book, log probability abstains	49.8	69.2	38.6	56.4
Closed book, probe abstains	49.7	69.2	37.1	56.0
Routed on token statistics	31.9	47.8	29.1	42.5
Always retrieve, probe abstains	29.9	48.3	25.2	43.3
Routed on activation probes	**29.0**	**47.2**	**23.1**	**40.7**

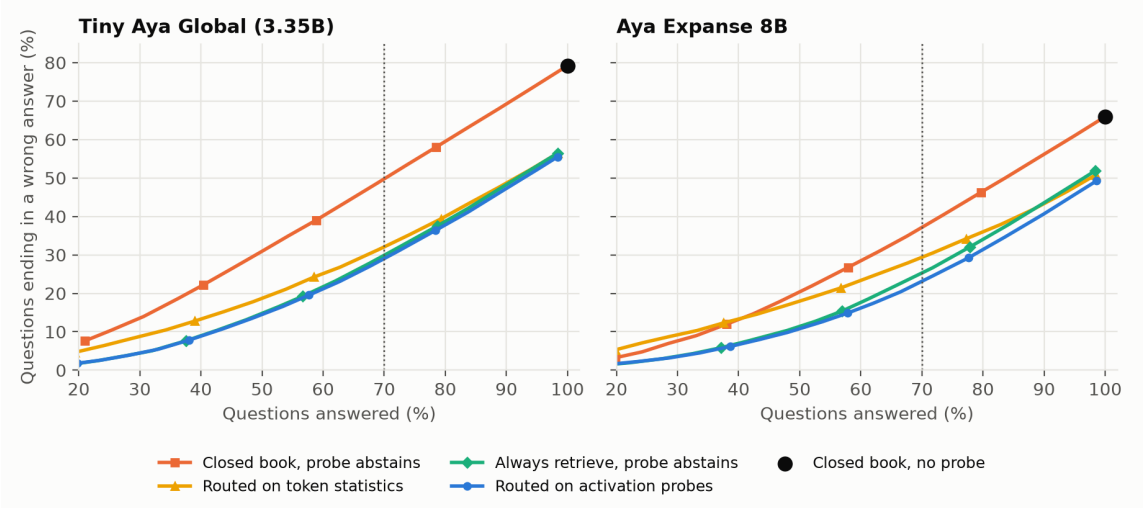


Figure 1: Wrong answers against questions answered, test split, both models.

This should not be oversold. At 70% answered, 41% of Tiny Aya's answers and 33% of the 8B model's are still wrong. These are small models on hard questions, and the system makes them much better without making them reliable.

3. What we built

Models. Tiny Aya Global (3.35B parameters) is the main model and Aya Expanse 8B is the replication. Both stay frozen. We chose a small multilingual model because it is the kind of model that gets deployed where a large one is too expensive, and it is the one that most needs to know when it does not know.

Data. NQ Open, TriviaQA, HotpotQA and PopQA, 16,320 questions after cleaning: 9,954 train, 2,398 validation and 3,968 test. They have gold answers that do not depend on another model's opinion, and they differ enough for one to be held out at a time. HotpotQA adds multi hop questions, and PopQA adds long tail facts with popularity data. We did not use TruthfulQA, HaluEval or RAGTruth, because their labels describe other models' outputs or free form misconceptions. CuratorKIT's exact and MinHash deduplicators ran over all datasets together with test items placed first, so a near duplicate was always removed from training rather than from test. Of 72,089 candidates, 65,774 survived, and we then sampled from those.

Labels. An answer is correct if a gold alias appears as a whole token span after normalization. A refusal counts as an abstention. An LLM based label audit of 180 train and validation answers, which was not a human audit, found no false positives among 41 labelled correct and 3 to 6% disputable among 139 labelled wrong. Measured accuracy is therefore slightly pessimistic.

Retrieval. Keyword search over English Wikipedia (the 20231101 dump), then a cross encoder (`ms-marco-MiniLM-L-6-v2`) reranks the top 100, and the best three passages go into the prompt.

Probes. L2 logistic regression on standardized hidden states, with temperature scaling. Layer, token position (last prompt token, last answer token, or the mean over answer tokens) and regularization strength are chosen on validation rows. Lee, Seo and Lim (2026) also find L2 logistic regression hard to beat, which matches what we saw when we tried fancier probes (Section 4.3).

System. Three probes share the work. One estimates whether the closed book answer is right. One reads the closed book pass and forecasts whether a retrieved answer would be right. One reads the retrieved answer and judges it. A question goes to retrieval when the forecast beats the closed book estimate, and the answer is withheld if confidence in it falls below a cut. For the utility tests, a value policy replaces the cut with expected value under stated costs: correct $+1$, wrong -2, abstain 0, retrieval -0.1, swept over wrong costs of 1, 2 and 4 and retrieval costs of 0.05, 0.1 and 0.2.

Discipline. The analysis plan, including every late change, is in `docs/analysis_plan.md`. Probes never saw test rows. The configuration was frozen in writing before the test split was first used, and it was run once. Two things changed after development, and we say so here. We moved the coverage result to the front, because that is what the problem statement asks for; the original primary test was utility. And some checks in Section 8 were added after the test run, and they are labelled as such.

4. How the work actually went

This section is the honest history. Everything here is also in `docs/progress_log.md` and `docs/experiment_registry.md`, with logs.

4.1 Things that were slow, and how we got past them

Decoding. Generation on the Mac was far slower than expected. We tested two explanations first, the generation wrapper and the attention mask, and both were wrong. The real cause was that Cohere's chat template adds a 361 token preamble to every prompt. Caching that preamble once gave a 3.8 times speedup, and a float32 test showed the outputs matched to rounding error.

Retrieval index. Our first index used SQLite full text search and took 14 to 107 seconds per query, which was unusable for 16,000 questions. We rebuilt it with Tantivy, which brought the median to 0.06 seconds.

Compute. A MacBook Air with 16 GB could not generate for both models in time. The 8B model was not practical there, and even Tiny Aya with retrieval started thrashing memory at batch 16. We moved generation to free Kaggle T4 pairs. The first 8B run with retrieval ran out of GPU memory after 6,784 rows, so the runner now splits a batch when it runs out of memory. In total the runs used about 7.5 hours of a T4 pair, not counting failed runs.

4.2 Bugs we found and fixed

Duplicate IDs. A duplicate ID bug let 1 test question and 12 validation questions overlap with other splits. We found it through a row count mismatch and fixed it with `scripts/finalize_splits.py`.

Refusals scored as wrong. While designing the policy code, we found that model refusals were being scored as wrong answers, even though the plan says a refusal is an abstention. We fixed it before any retrieved answer existed. The policy arithmetic and the AuditKIT scorer now both count refusals as abstentions, and the run stops if they disagree.

Wrong passages in one check. On the reranked runs, two scripts read the keyword passages instead of the reranked ones when checking whether the answer was in the passages. Policy decisions never used that column, but some relevance figures and the features of one exploratory combination did. We fixed it and reran all development analyses, keeping the earlier output.

Mismatched rows in a baseline. Tiny Aya's sampling baseline stopped at a time limit, so it covered fewer rows than the probe it was compared with. We rescored every comparison on shared rows only.

Comparisons at different coverage. On unseen datasets, a cut learned on other datasets lands each system at a different coverage, so an early version of our unseen gaps compared systems that answered different shares of questions. We replaced them with comparisons at exactly equal coverage (Section 8).

A significance test we could not use. AuditKIT 1.1.3's paired bootstrap has a known problem, tracked upstream as `https://github.com/Lexsi-Labs/AuditKIT/issues/20`. We kept AuditKIT for independent replay of decisions and wrote our own paired test, with unit tests for both false alarms and power.

An environment clash. Installing the demo's web library into the analysis environment silently downgraded pandas. We restored it, checked that all 53 tests pass and that a saved table reproduces exactly, and gave the demo its own environment.

4.3 Ideas that did not work

The value policy at the planned costs. The first end to end run showed that at the headline costs the value policy abstains on almost everything. Answering needs 67% confidence, which few answers reach. We kept the costs as planned rather than tuning them, and added the fixed coverage analysis, which is now the main result.

Fancier probes. An MLP probe scored lower than the linear one on both models. Adding a band of layers, token statistics or checks across the two paths made no reliable difference on development data. So the frozen system is the simple one.

Better passages beat cleverer routing. Error analysis on development data showed that the real limit was retrieval, not detection. With keyword passages, the model was right on 76% of questions when the answer was in the passages and on 6% when it was not. Adding the reranker raised answer presence in the top three from 37.7% to 54.3% on 300 validation questions, and on Tiny Aya it cut wrong answers at 70% answered from 35.4% to 29.0%. That is several times what routing adds.

Seeds as replication. We planned three probe seeds. They gave identical numbers, because the logistic fit is deterministic. We do not count them as replication, and we added a training subset check instead (Section 8).

Circuit analysis. We tried CircuitKIT's edge attribution patching on pairs of prompts with a helpful or misleading passage, to find components involved in using retrieved evidence. It was too slow on the Mac, and it ran out of GPU memory four times on a Kaggle T4 pair, even at batch 1, with shorter passages, with MLP only scope and with weight gradients switched off. We never found the cause, so there are no circuit results, and the four failed logs are kept.

5. Why these metrics

Wrong answer rate at fixed coverage is the problem statement in numbers. Macro AUROC, computed within each dataset and then averaged, stops a probe getting credit for merely telling easy datasets from hard ones. That matters, because activations predict which dataset a question

came from with 92 to 95% accuracy. ECE and Brier score show whether probabilities can be trusted as probabilities. Utility shows how a system does when errors and retrieval have prices. Intervals come from 2,000 resamples over questions, paired between systems.

6. Results

Confirmatory tests. Six tests per model were fixed before the test run, using the planned validation cut and Holm correction. Nine of twelve were confirmed, marked $\surd$ in Table 2.

Table 2: Routing minus rival, in points of wrong answers. $\surd$ marks a test confirmed after Holm correction.

Rival	Tiny 70%	Tiny 90%	8B 70%	8B 90%
Closed book probe	$-21.4\surd$	$-21.2\surd$	$-14.5\surd$	$-14.6\surd$
Always retrieve with probe	-0.8	$-1.3\surd$	$-1.7\surd$	$-2.4\surd$
Routing on token statistics	$-4.5\surd$	-0.8	$-7.5\surd$	-1.2

These tests use the coverage each system actually reached, which differed by up to 2 points in domain. At exactly equal coverage the gaps against token statistics are smaller, -2.9 and -6.1 at 70%, and Table 1 uses those equal coverage figures.

H1, unseen datasets: partly supported. Each dataset was held out in turn from probe training and selection. At exactly 70% answered, routing beats the closed book probe on all four unseen datasets, by 11 to 31 points on Tiny Aya and 6 to 26 points on the 8B model (Figure 2).

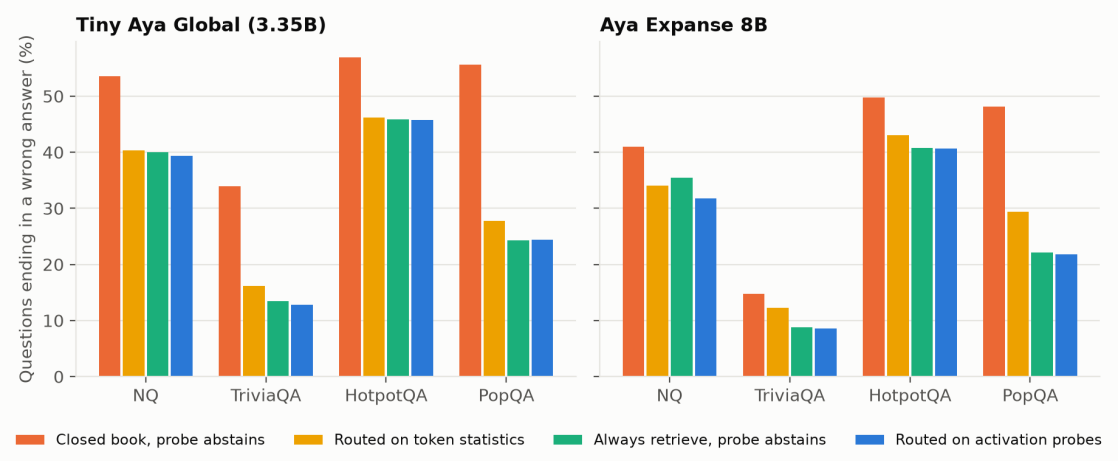


Figure 2: Wrong answers at exactly 70% answered on each dataset when it was left out of training.

Detection tells a more specific story (Figure 3, Table 3). For closed book answers on unseen datasets, activations have no consistent edge over a logistic model on token statistics: token statistics win on three of four datasets on Tiny Aya and on HotpotQA on the 8B model. After retrieval the picture changes. Token statistics fall to 0.69 to 0.77 on the 8B model while the probes hold 0.74 to 0.90, and activations win on all four datasets. On Tiny Aya they win on three and tie on HotpotQA. Our reading is that once a passage is in the prompt, the model can be fluent and confident about something it only copied, so output confidence says little while the hidden state still says something. That explanation is a hypothesis, since we did not test it directly.

In domain ECE for the probes is about 0.02. For retrieved answers on unseen datasets it rises to as much as 0.13, on TriviaQA and HotpotQA.

H2, decision quality: supported in domain, mixed on unseen data. The value policy beats the two threshold probe policy at the headline costs, by $+0.039$ utility on Tiny Aya (interval 0.029 to 0.049) and $+0.034$ on the 8B model (0.023 to 0.046), and it stays ahead at all nine cost settings.

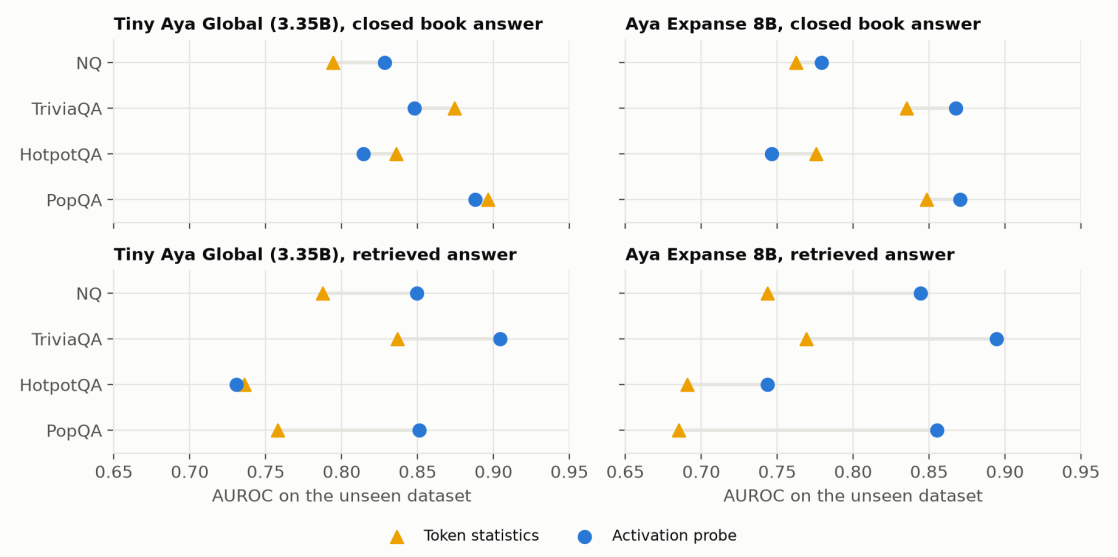


Figure 3: Macro AUROC on unseen datasets, activation probe against token statistics.

Table 3: Macro AUROC on the test split.

	Closed answer in domain	Retrieved answer in domain	Retrieved answer unseen range
Tiny Aya, activation probe	0.872	0.868	0.731 to 0.904
Tiny Aya, token statistics	0.854	0.782	0.736 to 0.837
Tiny Aya, mean log probability	0.840	0.732	0.687 to 0.770
Aya 8B, activation probe	0.852	0.877	0.744 to 0.894
Aya 8B, token statistics	0.808	0.724	0.685 to 0.769
Aya 8B, mean log probability	0.782	0.695	0.655 to 0.722

On unseen datasets it wins clearly on three of four for the 8B model and two of four for Tiny Aya. It does not beat the threshold policy on HotpotQA for either model, and on that dataset it scores below simply abstaining on everything (-0.022 and -0.090). Refitting the calibration on labelled rows from the held out dataset, as a diagnostic only, raises HotpotQA to -0.005 and -0.024, and TriviaQA on the 8B model from 0.273 to 0.366, so much of that loss is calibration rather than ranking. Utility at these costs is a strict test, since these policies answer only 15% and 26% of questions. The coverage results describe a usable system, and utility checks that the choices are sound when errors are expensive.

H3, retrieval efficiency: supported in a narrow sense. Routing retrieves for 85% of questions on Tiny Aya and 72% on the 8B model, with slightly fewer wrong answers than always retrieving at equal coverage in domain (0.9 to 2.7 points). On unseen datasets that accuracy edge mostly disappears, with only the 8B model on NQ clearly ahead at 70%, and retrieval rises as high as 95%. In a matched comparison without abstention, Tiny Aya matched always retrieving on correct answers with 85% retrieval. On the 8B model the cut chosen on validation fell slightly short on test (46.0% against 46.5% correct, at 48% retrieval). Mallen et al. (2022) note that small models need retrieval nearly always, and that is what we see. Most of the wrong answer reduction comes from retrieval plus the probe on the retrieved answer, not from the routing step.

Baselines. A probe on the last prompt token, before any answer exists, beats P(Yes) self rating on all folds: 0.839 against 0.580 on Tiny Aya and 0.806 against 0.610 on the 8B model. P(Yes) stands in for the prompted confidence of Query Level Uncertainty (Chen et al., 2026), though it is not their exact method. After generation, the activation probe beats agreement across five samples in domain (by 0.073 and 0.114 on shared rows) with no extra generation. On unseen datasets it

is clearly ahead on four of four for the 8B model and two of four for Tiny Aya, and the rest are inconclusive. Tiny Aya's samples stopped at a time limit, so its comparison covers 3,618 of 3,968 rows.

Table 4: Ablations: wrong answers as a percentage of all questions, in domain, at 70% answered unless noted.

	Tiny Aya	Aya 8B
Without probe (closed book, answer everything, 100% answered)	79.2	65.8
Without retrieval (closed book probe)	49.7	37.1
Log probability instead of probe	49.8	38.6
Token statistics instead of activations, routed	31.9	29.1
Without routing (always retrieve, probe)	29.9	25.2
Keyword search instead of reranking, routed	35.4	not run
Full system	29.0	23.1

With keyword passages at 90% answered, routing on token statistics beat the probe by 3.4 points. That is the one clear loss for activations among the routing comparisons.

7. Does it help in use

Compute per question from our logs, on Kaggle T4 pairs, is 0.11 s closed and 0.28 s with retrieval on Tiny Aya, and 0.13 s and 0.64 s on the 8B model. Reranking took 0.55 s per question on a MacBook Air, which is different hardware, so the sums in Table 5 are only a guide.

Table 5: Approximate seconds per question.

	Always retrieve	Routed
Tiny Aya	0.83	0.82
Aya 8B	1.19	0.98

Routing needs the closed book pass for every question, so it only saves compute when retrieval or generation with passages is expensive. It is about even on Tiny Aya and 18% cheaper on the 8B model. Where each retrieval has a real price, such as a paid search API, the 15 to 28% fewer calls count directly. The probes themselves cost one dot product, against five extra generations for self consistency.

We did not run a hosted closed model, so we make no price comparison with one. Our practical advice from these results is that the cheapest worthwhile step is to retrieve with a good reranker and put a probe on the retrieved answer. Routing is worth adding where retrieval calls cost money.

8. Checks

Is the probe reading popularity? This check was planned. On PopQA, page views alone reach 0.60 to 0.66 AUROC on closed book answers and about 0.46 after retrieval, while the probe holds 0.87 to 0.94 inside every popularity quartile in domain, and 0.81 to 0.91 with PopQA held out. So the probe is not just reading how famous the subject is. Quartiles hold about 250 questions each, so differences between them are noisy.

Training rows (added after the test run). Refitting the frozen probes on five random 90% subsets of training rows moved each system's wrong answer rate by less than half a point. Routing's lead over the closed book probe stayed between 20 and 22 points on Tiny Aya and between 14 and 15 on the 8B model at 70% and 90%. Its lead over always retrieving on Tiny Aya at 70% ranged from -1.1 to $+0.2$, which agrees with that test not being confirmed.

Exact coverage (added after the test run). All unseen comparisons in this report set each system's cut from its own confidence on the evaluation rows. This uses no labels, but it needs the whole unlabelled batch, so it compares ranking, not a system you could deploy unchanged. With the planned cut, a 70% target reached between 56% and 83% on unseen datasets, which is a real limitation.

Online cut (added after the test run). The drift above has a simple label free fix. Each question's cut is set from the confidence of the previous 100 questions, without looking at the question being decided. The window size was chosen on validation data by a rule written down before any numbers were seen. We tested a stream that starts on the training datasets and then switches to the unseen one, in 20 random orders. With a 70% target, the four systems answered between 67.7% and 71.1% of questions on every unseen dataset, against 46.7% to 94.0% with the planned cut. With a 90% target they answered 87.1% to 90.0%, against 78.4% to 99.6%. Wrong answer rates came within 1.3 points of the exact batch cut, so the exact coverage comparisons above describe a system that can actually be deployed. It still needs about 100 questions from a new domain before the cut settles.

Other languages (added after the test run). The probes were trained on English only, while the Aya models are built to be multilingual, so we tested whether they transfer. We took 600 MKQA questions (Longpre et al., 2021) in English, Spanish, German, French, Russian and Turkish, keeping only name and short phrase answers and dropping any question already in our set. Only the question was translated; the prompt template stayed the same. Answers were generated closed book on Kaggle T4 pairs, with weights checked file by file against the Hugging Face revision used everywhere else, and the frozen in domain probe was applied with no refit. The analysis was written into the plan before generation.

Table 6: Closed book answers on the same 600 MKQA questions per language. AUROC of the frozen English trained probe against a token statistics model.

	Tiny Aya			Aya 8B		
Language	Correct	Probe	Token stats	Correct	Probe	Token stats
English	22.8%	0.872	0.857	48.3%	0.861	0.817
Spanish	20.5%	0.862	0.825	43.2%	0.843	0.774
German	20.0%	0.857	0.859	41.7%	0.838	0.775
French	20.0%	0.862	0.841	42.0%	0.841	0.764
Russian	13.5%	0.907	0.859	28.8%	0.836	0.784
Turkish	17.7%	0.845	0.831	34.8%	0.846	0.784

On both models the probe's AUROC stays within 0.035 of English in every language, and every gap's 95% interval includes zero. Token statistics transfer less well on the 8B model, falling to 0.76 to 0.78 outside English while the probe holds 0.84 to 0.85. On Tiny Aya the two are closer. Accuracy drops outside English, most for Russian, so a working check matters more there: with the online cut at a 70% target, Aya 8B leaves 24.8% of questions wrong in English and 28.7% to 40.8% elsewhere, against 51.5% to 71.2% when answering everything. Tiny Aya often answered foreign questions in English, which is why English gold answers count. Our refusal patterns are English only, so a refusal in another language counts as a wrong answer, though detected refusals were under 0.5% everywhere. Retrieval and the routed system were not tested outside English.

Independent replay. Every policy's final answers were replayed through AuditKIT with a separate scorer, and the run stops if its utility differs from ours.

Live path. SafeTune's `LinearProbeGuardModel` ran the prompt only probe live on 200 validation questions with Tiny Aya on a Mac. Flagged prompts were sent to retrieval through its `on_refuse` hook in place of a refusal. Its scores matched our saved T4 activations (rank correlation 0.9999, identical flags), and answer labels matched the saved runs on 99.5% of questions. This is an

integration test, not a result.

9. Toolkits

CuratorKIT (Bhattacharjee et al., 2026) cleans and deduplicates the data, SafeTune's probe guard runs the probe at inference time, and AuditKIT replays every decision independently. CircuitKIT was attempted and did not fit in memory, as described above.

10. Related work and what is new here

Predicting errors from activations is well established (Azaria and Mitchell, 2023; Kossen et al., 2024; Orgad et al., 2024), and so is using internal state to trigger retrieval (Yao et al., 2024; Liu et al., 2024; Baek et al., 2024). SABER (2026) is the closest work: it uses hidden state probes to trust the parametric answer, trust the retrieved answer or abstain, with leave one dataset out tests, but it assumes the context is already supplied, so it does not address whether to retrieve. Query Level Uncertainty (Chen et al., 2026) decides on retrieval before generation using a training free signal, without pricing retrieval or abstaining. We did not run SABER and compare with it on design only. Sengupta, Seth and Sankarapu (2025) argue that internal understanding should be part of how an aligned system is built, not only a diagnostic run afterwards. A probe that makes the answer, retrieve or abstain decision at inference time is one small, concrete case of that.

Among the papers we read, we did not find one that combines a trained activation forecast of how the retrieved answer would do, a priced retrieval decision, a second decision after retrieval, and wrong answer rates at fixed coverage, with token statistics put through the same pipeline as a control. Our search was limited, and this is an integration and measurement contribution, not a new probing method. The finding we would most like others to check is that the advantage of activations over token statistics shows up mainly after retrieval.

11. Limitations

We used two models from one family. Outside English we tested only closed book answers in five languages, all written with spaces between words, so retrieval, routing and other scripts remain untested there. Labels come from string matching, and the label audit was LLM based, not human. Questions are short and factual, so nothing here carries over to long form generation. Retrieval uses one Wikipedia dump, and some NQ gold answers have aged. The Tiny Aya sampling baseline is incomplete, and the 8B keyword run did not finish, so that ablation is Tiny Aya only. A cut learned on other datasets drifts on a new one; the online cut fixes this, but only after about 100 questions. HotpotQA is a weak spot for the probe on retrieved answers on both models, at about 0.74. The answers the system does give are still wrong too often for high stakes use.

12. Reproducing this

See the README. Every number here traces to a CSV in `results/` and a log named in `docs/experiment_registry.md`. Failed runs are kept in the registry and their logs are in the repository.

References

- Azaria and Mitchell (2023). The internal state of an LLM knows when it's lying. arXiv 2304.13734.
- Baek et al. (2024). Probing-RAG. arXiv 2410.13339.
- Bhattacharjee et al. (2026). CuratorKIT. arXiv 2606.21631.
- Chen, de Melo, Suchanek and Varoquaux (2026). Query-Level Uncertainty. ICLR 2026, arXiv 2506.09669.

- Kossen et al. (2024). Semantic Entropy Probes. arXiv 2406.15927.
- Lee, Seo and Lim (2026). The Hallucination Signal Is a Mean Shift. arXiv 2608.28930.
- Liu et al. (2024). CtrlA. arXiv 2405.18727.
- Longpre, Lu and Daiber (2021). MKQA. TACL, arXiv 2007.15207.
- Mallen et al. (2022). When not to trust language models. arXiv 2212.10511.
- Orgad et al. (2024). LLMs know more than they show. arXiv 2410.02707.
- SABER (2026). Trust or Abstain? A Self-Aware RAG Approach. arXiv 2605.18792.
- Sengupta, Seth and Sankarapu (2025). Interpretability as Alignment. arXiv 2509.08592.
- Yao et al. (2024). SeaKR. arXiv 2406.19215.